FaCTS — Review 2 Summary: Data, Plain QLoRA Baseline, Preliminary Evaluation

Project: *Fact-Constrained, Traceable Summarization of clinical doctor–patient conversations* (BCSE306L). Review 2 delivers the three foundations the report defines for the baseline stage; the FaCTS model itself (fact-table conditioning, ORPO factuality preference training, verifier / repair) is Review 3 and is deliberately not started.

role	delivered	documentation
Person 1 — data	3 corpora inspected and standardised (12,108 records); cleaning, duplicate / near-duplicate / leakage audit; train 4,265 / val 220 / test 770 with SHA-256 manifest; lengthy-conversation policy from a Llama-token audit; **rule-based clinical fact extractor** (symptoms, history, diagnosis, medications, dosages, investigations, treatment / follow-up, negation) and a facts dataset for every record	`docs/PERSON1_DATA_PIPELINE.md`
Person 2 — model	Llama-3.1-8B-Instruct selected; 4-bit NF4 QLoRA (r=16, 41.9 M trainable params) trained on Kaggle 2×T4 (332 steps, best eval loss 0.9555); 9 GPU / memory / pipeline issues solved; validation tests + zero-shot control; 770 test notes generated under two decoding configurations; adapter, checkpoints and full config saved and sha-pinned; handoff completed	`docs/PERSON2_MODEL_TRAINING.md`, `REVIEW2_BASELINE.md`, `STATUS.md`
Person 3 — evaluation	ROUGE-1/2/L/Lsum, BERTScore; factuality checker for the six error types of the report; hallucination rate, negation-flip rate, numerical consistency, clinical-information coverage; tables, figures, example errors; preliminary results and conclusions	`docs/PERSON3_EVALUATION.md`

Submission PDF: `reports/Review2_Documentation.pdf` (this summary + the three role documents + the generated reports). Code: `src/cfe/`, `src/eval/`, `scripts/`, `kaggle/`; 22 unit tests pass.

Headline numbers (plain QLoRA baseline, five official test splits pooled, n = 520, ×100)

system	ROUGE-L ↑	BERTScore ↑	hallucination rate ↓	negation-flip rate ↓	numerical consistency ↑	clinical coverage ↑
baseline, run 1 decoding (primary)	**30.7**	ACI −1.8 / MTS 39	**40.4**	**8.2**	**71.1**	**61.3**
baseline, run 3 decoding (anti-repetition)	25.1	ACI −15 / MTS 31	59.3	10.7	72.0	54.0
baseline run 1, clean outputs only (n = 416)	35.1	—	33.8	5.0	77.4	69.3
zero-shot Llama-3.1-8B-Instruct, validation (n = 20)	21.2	16.2	18.0	14.4	73.4	71.6

Per split (run 1): ACI-Bench full notes ROUGE-L 17.4–17.9 (72–78 % of outputs truncated), MTS-Dialog sections 35.8 / 33.5, DialogueSOAP (run 3, synthetic) 29.7.

Five findings the Review 3 design rests on

1. **The baseline writes sections well and full notes badly.** MTS-Dialog ROUGE-L (35.8 / 33.5) is in the range of published MEDIQA-Chat 2023 systems; ACI-Bench (17.4–17.9, published ~40–45) is dragged down by 72–78 % truncated outputs. On the clean ACI-Bench outputs ROUGE-L is 29.6–31.2 — the model can write the note but has not learnt to end it: only 67 of 4,265 training examples (1.6 %) are full notes and the best checkpoint is at 0.75 epoch. Fix for Review 3: up-weight ACI-Bench, complete the 2 epochs.
2. **Decoding heuristics trade repetition for hallucination.** Raising `repetition_penalty` to 1.2 and blocking repeated 6-grams removed the loops and made every metric worse (ACI-Bench hallucination 55 → 78 %, ROUGE-L 17.9 → 11.4): the model replaced grounded repetition with invented findings. Repetition control is not a factuality tool; grounding must come from the input (the fact table).
3. **Fine-tuning made the model less faithful than the un-tuned base.** On the same validation items the zero-shot base hallucinates 18 % of entities and covers 72 % of reference facts; the adapter 42–73 % and 40–50 %. The adapter learnt the *form* of a note faster than the discipline of writing only what was said — the exact gap that fact-conditioning and ORPO target. The zero-shot row stays as a control in every Review 3 table.

4 **Medications and investigations are the information the baseline loses** (coverage 40–48 % and 30–37 % on ACI-Bench vs 55–59 % for symptoms); 23–27 of 40 ACI-Bench notes omit at least one medication the reference records, and numbers are unreliable (numerical consistency 55–61 % on ACI-Bench: invented lab values, changed durations).

5 **Standard and factuality metrics disagree by design**: ROUGE-L rewards a runaway symptom list that overlaps the reference vocabulary, the hallucination rate separates clean from degenerate outputs (39 % vs 90 %). Both are reported in every table; neither replaces the other.

What the data work established

- Official splits kept; leakage check between the training mix and the 770 test items: 0 id overlap, 0 exact dialogue overlap (3 recurring two-line MTS-Dialog exchanges dropped from train), 2 near-duplicate dialogues that reuse one MTS-Dialog conversation for another section target (flagged, not removed).
- 131 one-word MTS-Dialog targets (`None.`, `Noncontributory.`) identified and kept on purpose.
- Token-length audit with the training tokenizer: ACI-Bench prompt+note p99 = 4,129 tokens, so the 4,096-token window drops exactly one training example; test prompts are never cut at inference.
- Fact extractor: 87.5 % fully-correct facts on a 40-fact manual sample (95 % correct entity + polarity), 82–85 % of reference-note entities also found in the transcript, ~130 ms per encounter on CPU.

Honest limitations

- Kaggle 2×T4: 1.24 of 2 planned epochs; one training run (a second would cost ~11 h of the 30 h/week quota).
- The factuality checker is lexicon-based (no UMLS); rates carry ±5 points of extraction noise and are *preliminary*. Human evaluation is Review 3.
- DialogueSOAP is synthetic and is reported, not headlined. The run-2 output (wrong adapter) is excluded.

Reproduce everything (laptop CPU unless noted)

```
python scripts/import_datasets.py && python scripts/make_baseline_split.py
python scripts/audit_dataset.py                      # reports/data_audit/
python scripts/extract_clinical_facts.py             # data/processed/facts/
kaggle kernels push -p kaggle/baseline --accelerator NvidiaTeslaT4 && python scripts/fetch_kaggle_run.py   # GPU
python -m src.eval.run_eval --run outputs/kaggle_run1_old_decoding/outputs/baseline_qlora_llama31_8b --label run1_rep1.1 --bertscore
python -m src.eval.run_eval --run outputs/kaggle/outputs/baseline_qlora_llama31_8b --label run3_rep1.2_ngram6 --bertscore
python -m src.eval.run_eval --compare && python -m pytest tests -q
python scripts/build_docs_pdf.py                     # reports/Review2_Documentation.pdf
```

Person 1 — Data Pipeline and Clinical Fact Extraction (Review 2)

Owner: data role. Scope: everything between the raw downloads and the training-ready dataset, plus the *basic* clinical fact extractor the evaluation (Person 3) and Review 3 build on.

#	Task	Where it is done	Section
1	Inspect the downloaded datasets	`scripts/download_datasets.py --list`, `scripts/import_datasets.py --validate-only`	§1
2	Understand the structure, select the dataset(s)	§1–§2	
3	Clean the doctor–patient conversations	`scripts/import_datasets.py` (+ audit)	§3
4	Standardize the dataset format	`scripts/import_datasets.py` → one JSONL schema	§3
5	Remove duplicates, missing / unusable samples, check leakage	`scripts/make_baseline_split.py`, `scripts/audit_dataset.py`	§4
6	Prepare train / validation / test datasets	`scripts/make_baseline_split.py`	§5
7	Handle lengthy conversations	token audit in `scripts/audit_dataset.py`; policy in §6	§6
8	Basic clinical fact extraction (8 categories incl. negation)	`src/cfe/` + `scripts/extract_clinical_facts.py`	§7
9	Final training-ready dataset + clinical fact data for Person 2	`data/processed/baseline_*.jsonl`, `data/processed/facts/`	§8

Every number in this document is produced by a script and stored in `reports/data_audit/data_audit.json` / `data/interim/cfe_stats.json`; re-running the commands in §9 regenerates them.

---

1. Dataset inspection

Three dialogue → note corpora were downloaded (`data/raw/`, registry and SHA-256s in `data/MANIFEST.json`). Raw formats differ in every respect, which is why an importer exists:

corpus	raw format	pairs	what one record is	licence
ACI-Bench (Yim et al., 2023)	JSON `{"data":[{"src","tgt","file"}]}`, 5 files	67 / 20 / 40+40+40	a *full* recorded encounter (`[doctor]` … `[patient]` …, ~1,300 words, ~50 turns) → a complete clinical note (CC / HPI / ROS / PE / RESULTS / A&P, ~420 words)	CC-BY 4.0
MTS-Dialog (Ben Abacha et al., 2023)	CSV with embedded newlines, 4 files	1,201 / 100 / 200+200	a *short* dialogue snippet (`Doctor:` … `Patient:` …, ~65 words, ~6 turns) → **one section** of a note, labelled with a 20-value `section_header` (GENHX, FAM/SOCHX, PASTMEDICALHX, CC, ROS, MEDICATIONS, …)	CC-BY 4.0
DialogueSOAP (omi-health, HF)	JSON **Lines** (not a JSON array), 3 files	9,250 / 500 / 250	a synthetic dialogue (~420 words, ~17 turns) generated by an LLM from a PMC case report → a SOAP note (`S:`/`O:`/`A:`/`P:`, ~245 words)	MIT (synthetic)

Observed quirks that shaped the cleaning:

- ACI-Bench transcripts are ASR-style: lower-case, tokenised punctuation (`hi , andrew .`), speaker tags in brackets, occasionally `[patient_guest]`. Notes are ALL-CAPS section headers on their own line.
- MTS-Dialog has 131 one-word targets in train (`None.`, `Noncontributory.`, `Asthma.`) — legitimate section values, not corruption — and a handful of two-line "any known drug allergies? / No." dialogues that recur verbatim across its official splits.
- DialogueSOAP notes are long, dense and LLM-written; its `prompt` field is the generation prompt of the original authors and is kept in `metadata` but never shown to our model.

2. Dataset selection and role

All three are used, with distinct roles fixed by the project report:

corpus	role	reason
ACI-Bench	**primary target task, headline test set**	the only corpus with real full encounters and complete notes; the FaCTS claim (fewer hallucinations at equal ROUGE) has to be shown here
MTS-Dialog	section-level supervision, second test set	in-domain, human-written, same provenance as ACI-Bench (MEDIQA-Chat 2023); teaches section writing cheaply

corpus	role	reason
DialogueSOAP	training volume only	synthetic; the report forbids using it as the sole evaluation basis, so its test split is *reported* but is not a headline number

3. Cleaning and standardization

`scripts/import_datasets.py` converts each corpus into one schema:

```
{"id": "aci_bench_train_D2N002-virtassist", "dataset": "aci_bench", "split": "train",
 "source_dialogue": "...", "target_summary": "...", "metadata": {"file_id": "D2N002-virtassist"}}
```

Cleaning applied per record (identical for all corpora):

1. **Field mapping**: `src/tgt`, `dialogue/section_text`, `dialogue/soap` → `source_dialogue` / `target_summary`. Corpus-specific metadata is preserved (`file_id`; `row_id` + `section_header`; `prompt`) so nothing is lost and the MTS section is available as a task instruction.
2. **Whitespace normalisation**: outer `strip()`; internal newlines kept (turn boundaries and section breaks are meaning-bearing for both the model and the fact extractor). Speaker tags are *not* rewritten — the model must learn the real ASR style of the target task.
3. **Empty / null drop**: any record with an empty source or empty target is skipped and counted (`skipped` in the importer summary). None were found in the three corpora.
4. **Encoding**: everything is read and written as UTF-8; the audit scans for control characters (none found).
5. **Stable ids**: `<dataset>_<split>_<native id>` so every prediction, fact and metric can be joined back to its record without positional indexing.

Official splits are kept as delivered — nothing is re-split, so results stay comparable to the published leaderboards.

4. Duplicates, unusable samples, leakage

`scripts/audit_dataset.py` (results: `reports/data_audit/data_audit.md`).

Exact duplicates. Normalised (whitespace-collapsed, lower-cased) source dialogues: 2 duplicate sources inside MTS-Dialog train, 1 in test_sum_2023, none elsewhere. Duplicate *pairs* (same source *and* target): 1 in MTS-Dialog train. Duplicate ids: 0.

Near-duplicates. 64-bit SimHash over 3-word shingles, banded candidate search, Hamming distance ≤ 3. ACI-Bench: 0 pairs. DialogueSOAP: 0 pairs (9,250 synthetic dialogues are all distinct). MTS-Dialog: 12 pairs, 3 within a split and 9 across splits — every one is the MTS-Dialog design of re-using one dialogue for several section targets, listed in `reports/data_audit/near_duplicates.jsonl`.

Unusable samples. Criteria: empty text, source < 5 words, target < 2 words, target identical to source, control characters. Only the "target < 2 words" rule fires: 131 / 10 / 14 / 13 records in the four MTS-Dialog splits, all one-word section values (`None.`, `Penicillin.`, `Noncontributory.`). **Decision: keep them.** They are correct outputs for their section (writing `None.` under MEDICATIONS when the patient takes nothing is the desired behaviour), the official test sets contain the same kind of item, and dropping them from train would bias the model towards always producing content. They are flagged in `data_audit.json` so the evaluation can report metrics with and without them.

Leakage. Checked between the baseline mix (`baseline_train.jsonl` ∪ `baseline_val.jsonl`) and all six test splits:

check	result
id overlap train/val, train/test, val/test	0 / 0 / 0
exact source-dialogue overlap mix ↔ test	**0** after `make_baseline_split.py` dropped 3 MTS-Dialog train rows (`mts_dialog_train_901/926/989`, the recurring "any known drug allergies? / No." exchange)
exact target overlap mix ↔ test	131 — all boilerplate one-word/one-line targets (`None.`, `No known drug allergies.`); not content leakage
near-duplicate sources mix ↔ test (Hamming ≤ 3)	2: `mts_dialog_train_318`↔`test_chat_2023_96` (punctuation variant of the allergy exchange) and `mts_dialog_train_543`↔`test_sum_2023_78` (same dialogue, different section target)
`train_manifest.json` SHA-256 vs files on disk	match (train and val)

The two near-duplicate test ids are flagged in the evaluation (`FLAGGED_TEST_IDS` in `src/eval/run_eval.py`); their effect on a 400-item MTS test set is below reporting precision, and the official test sets are not modified.

5. Train / validation / test preparation

`scripts/make_baseline_split.py` (seed 42) builds the Review 2 mix specified in `REVIEW2_BASELINE.md` §4.1 and writes `train_manifest.json` with counts and SHA-256s:

file	n	composition
`baseline_train.jsonl`	**4,265**	ACI-Bench train 67 + MTS-Dialog train 1,198 (1,201 − 3 dropped) + DialogueSOAP train 3,000 (random, seed 42)
`baseline_val.jsonl`	**220**	ACI-Bench val 20 + MTS-Dialog val 100 + DialogueSOAP val 100
test (untouched, 6 files)	**770**	ACI-Bench 40+40+40, MTS-Dialog 200+200, DialogueSOAP 250

Why 3,000 of the 9,250 DialogueSOAP rows: enough to teach SOAP structure, while keeping the 2-epoch run inside a Kaggle 2×T4 budget; the full set triples training time for a synthetic corpus. Records are shuffled once after mixing so that `group_by_length` batching in the trainer is the only ordering effect. The same files were uploaded unchanged as the private Kaggle dataset `hirthick17/medsum-fact-processed` and are what the adapter was trained on (SHA-256 verified by the Kaggle kernel and by the audit).

6. Lengthy conversations

Lengths were measured in **Llama-3.1 tokens exactly as the trainer sees them** (system prompt + task instruction + transcript rendered with the chat template, plus the note and the end-of-turn token), using the tokenizer saved with the adapter:

corpus	prompt tokens p50 / p95 / max	prompt + note p50 / p99 / max	> 4,096 (dropped from training)
ACI-Bench train	1,619 / 2,411 / 3,635	2,195 / 4,129 / 4,405	1 (`D2N021-virtscribe`)
ACI-Bench test (3 splits)	1,429–1,641 / 2,402–3,028 / 3,275	1,938–2,224 / 3,913–4,312 / 4,461	0 at inference (prompt only)
MTS-Dialog train	169 / 533 / 2,254	193 / 1,157 / 3,681	0
DialogueSOAP train	688 / 794 / 946	1,050 / 1,292 / 1,491	0
baseline mix	—	992 / 1,991 / 4,405	**1 of 4,265**

Policy, in order of what was considered:

1. **Truncate the transcript** — rejected: the reference note still asserts facts from the cut part, so the model would be trained to hallucinate.
2. **Truncate the note** — rejected: the model learns to stop mid-note (exactly the failure mode we later measured as run-1 truncation).
3. **Chunk the dialogue** and summarise per chunk — deferred to Review 3 (it needs the fact table to merge chunks coherently; MODEL_PLAN.md Stage 0/1).
4. **Drop over-length training examples and keep the full context at inference** — adopted. The training window is 4,096 tokens (the largest that fits 2×T4 with the fused cross-entropy, REVIEW2_BASELINE.md §3.4); exactly one training example exceeds it. At inference the prompt is never cut (Llama-3.1 context is 128k), so all 770 test conversations are seen in full.

The prompt share of a training example is 0.73 (ACI-Bench) / 0.86 (MTS-Dialog) / 0.65 (DialogueSOAP): this is why loss is computed on the note tokens only (§4.4 of the baseline doc); otherwise ~75% of the gradient would be spent learning to reproduce transcripts.

7. Basic clinical fact extraction (`src/cfe/`)

7.1 Design

A deterministic, dependency-free extractor that turns any transcript or note into a list of **facts** with provenance. It is the Review 2 stand-in for the scispaCy + UMLS Stage 1 of MODEL_PLAN.md and keeps the same schema, so the evaluation and the Review 3 prompt builder do not change when the extractor is upgraded.

```
Fact = {category, text, canonical, polarity, certainty, temporality, experiencer,
        section, speaker, start, end, sentence_idx, value, unit, frequency, route, drug, subtype, trigger, is_class}
```

required category	how it is recognised	canonical form
Symptoms	146 canonical symptoms / signs, 684 surface forms (`short of breath`, `sob`, `can't breathe` → `dyspnea`)	lexicon canonical

required category	how it is recognised	canonical form
Medical history	a condition mention in a history context: PMH / family / social section, or a historical trigger (`history of`, `years ago`, `known`, `s/p`), or a family experiencer; drug allergies (ALLERGIES section, `allergic to X`)	condition canonical (237 conditions, 1,037 surfaces)
Diagnosis	any other condition mention; `subtype="asserted"` when it sits in ASSESSMENT/PLAN/CC or next to a diagnostic cue (`diagnosed with`, `consistent with`, `likely`, …), otherwise `"mentioned"`	condition canonical
Medications	357 generics with US brand names (`Lipitor` → `atorvastatin`) + 37 drug classes flagged `is_class` (`antibiotic`, `statin`)	generic name
Dosages	regex `number + unit` (mg, mcg, mL, units, puffs, tablets, %, …) with optional frequency (`twice a day`, `q6h`, `prn`) and route (`po`, `IV`, `topical`), attached to the nearest drug in the sentence (preceding within 80 chars, else following within 40)	`<drug> <value><unit>`
Investigations	113 canonical labs / imaging / exam manoeuvres / scores, 1,143 surfaces (`a1c`, `hba1c`, `glycated hemoglobin` → `hemoglobin a1c`); vitals and lab **values** become `numeric` facts with `subtype = vital / lab / age / duration`	lexicon canonical
Treatment / follow-up	18 canonical treatment types (surgery, injection, physical therapy plan, wound care, lifestyle modification, referral, hospital admission, discharge, medication management, …) with 1,671 surfaces, plus a regex for follow-up *timing* (`follow up in 4 weeks` → value 4, unit week)	canonical + value
Negations	NegEx/ConText-style assertion on every fact: pre-triggers (`denies`, `no`, `without`, `negative for`, `don't think`, …), post-triggers (`was ruled out`, `resolved`), pseudo-negations masked (`no increase`, `not only`), scope closed by `but / however / except`, 12-word window; plus **dialogue Q/A rule**: a bare `No.` from the patient negates the terms asked in the doctor's question sentence, a bare `Yes.` confirms them	`polarity = absent`

Additional assertion attributes come for free from the same rule set: `certainty` (`possible`, `rule out`, `?`, `side effects can include …` → uncertain), `temporality`, `experiencer`.

Text handling: the segmenter detects dialogue vs. note, splits dialogue into speaker turns (`[doctor]`, `Doctor:`, `Guest_family:`), notes into sections (ALL-CAPS headers, `S:/O:/A:/P:`, `HEADER: text`; MTS-Dialog sections come from `section_header` via `MTS_SECTION_MAP`), then into sentences with character offsets. Lexicon matching is longest-match-wins across categories; abbreviations of ≤ 3 letters (`CAD`, `CHF`, `DOE`) must appear in upper case so `as`, `pe`, `ms` in prose never fire; short terms take no plural suffix.

7.2 Validation

- **Unit tests**: `tests/test_cfe.py` (14 tests: negation scope, pseudo-negation, post-negation, the dialogue Q/A rule, dosage/frequency attachment, sections and routing, vitals/labs/ages/durations, follow-up timing, provenance offsets, abbreviation casing, class flags) — all pass.
- **Manual spot-check**: 40 randomly sampled facts from ACI-Bench test reference notes (`reports/data_audit/cfe_samples.md` has the rendered samples). Before the fixes: 35/40 fully correct, 38/40 correct at entity+polarity level. The five errors (a `DOE` match inside "does", a drug in the ALLERGIES section counted as a medication, `9-month-old` given unit *years*, a hypothetical side-effect listed as a present symptom, `has had` marking a current complaint as historical) led to the abbreviation-plural, allergy-section, age-unit and hypothetical-cue rules now in the code; the first four are fixed, the temporality one is a known limitation.
- **Corpus-level sanity** (`reports/data_audit/cfe_stats.md`): 82–85% of the entities extracted from an ACI-Bench reference note are also extracted from its transcript (72–79% for MTS-Dialog, whose notes are written from a wider context than the snippet). Negation rate of reference-note facts is 13–14% (ACI-Bench, ROS-heavy) / 18–26% (MTS-Dialog) / 6% (DialogueSOAP), matching the styles of the corpora. Dosage facts are linked to a drug name 83–93% of the time in ACI-Bench notes.
- **Speed**: 120–150 ms per ACI-Bench record (dialogue + note), ~10 ms per MTS-Dialog record; the full 12,108-record corpus takes ~9 minutes on the laptop CPU.

7.3 Known limitations (carried to Review 3)

Lexicon recall is bounded by the lists (no UMLS linking, no unseen drug names); multi-word symptom phrasing outside the lists falls back to the generic `pain`/`swelling` entry; temporality from `has had` over-fires on current complaints; no

coreference (`it hurts` is not linked to the knee). Every one of these is addressed by the scispaCy + UMLS + negspaCy Stage 1 planned for Review 3, which slots in behind the same `Fact` schema.

8. Deliverables handed to Person 2 and Person 3

file	content	consumer
`data/processed/<corpus>/<split>.jsonl` (12 files)	unified, cleaned records	everyone
`data/processed/baseline_train.jsonl`, `baseline_val.jsonl`, `train_manifest.json`	the training-ready mix, SHA-256-pinned	Person 2 (trained on exactly these; the Kaggle kernel checks the SHA)
`data/processed/facts/<corpus>_<split>.jsonl` (12 files, 311 MB)	per record: `source_facts`, `target_facts`, per-category summaries	Person 3 (factuality metrics), Review 3 (fact-table prompting)
`data/interim/cfe_stats.json`, `reports/data_audit/cfe_stats.md`, `cfe_samples.md`	extractor statistics and manual-check samples	documentation
`reports/data_audit/data_audit.{json,md}`, `length_hist.png`, `near_duplicates.jsonl`	the audit in §4–§6	documentation

9. Reproduce

```
python scripts/download_datasets.py --tier core --tier extra   # ~1.3 GB
python scripts/import_datasets.py                               # -> data/processed/<corpus>/<split>.jsonl
python scripts/make_baseline_split.py                           # -> baseline_train/val.jsonl + train_manifest.json
python scripts/audit_dataset.py                                 # -> reports/data_audit/  (2 min; --no-tokens for 20 s)
python scripts/extract_clinical_facts.py                        # -> data/processed/facts/ (8 min)
python -m pytest tests -q                                       # 22 tests
```

Person 2 — Plain QLoRA Baseline: Model Selection, Training, Inference, Handoff (Review 2)

Owner: model-training role. Scope: the **plain QLoRA baseline** only (the report defines this model as the experimental baseline; FaCTS fact-conditioning, ORPO and the verifier are Review 3).

This document is the *record* of what was done and what came out. The full step-by-step procedure, with code and the troubleshooting table, is `REVIEW2_BASELINE.md` (§1–§10); the run-by-run narrative of what went wrong and how it was fixed is `STATUS.md`. Both are referenced by section below rather than repeated.

#	Task	Outcome	Where
1	Select the open-source LLM	Llama-3.1-8B-Instruct (ungated mirror `unsloth/Meta-Llama-3.1-8B-Instruct`); BioMistral-7B kept as fallback, not needed	§1, REVIEW2_BASELINE §1
2	Python / HF / PyTorch environment	Kaggle 2×T4 driven headlessly from the laptop with the Kaggle CLI; versions pinned in `training_config.json`	§2, REVIEW2_BASELINE §2
3	4-bit QLoRA setup	NF4 + double-quant, fp16 compute (T4 has no bf16), LoRA r=16 α=32 on all 7 projections, 41.9 M trainable / 8.07 B (0.52%)	§3, REVIEW2_BASELINE §3
4	Training pipeline	`kaggle/baseline/baseline_qlora.py`: chat-template prompts, assistant-only loss, length audit, leakage check, Liger fused cross-entropy, 2-GPU naive pipeline, wall-clock guard	§3, REVIEW2_BASELINE §4
5	Fine-tune the baseline	332 optimizer steps / 1.24 epochs on 4,265 examples, 7.5 h; best eval loss 0.9555 at step 200 → adapter `final/` (sha256 `14e71261…`)	§4
6	Solve model / GPU / memory / training issues	9 distinct issues hit and fixed (OOM in cross-entropy, OOM in backward, cross-device loss, P100 incompatibility, bitsandbytes/triton drift, 12-h cap, lost log, wrong adapter mounted, repetition loops)	§5, REVIEW2_BASELINE §6, STATUS
7	Test the trained model	validation checks in a fresh process (adapter reload = the test itself), zero-shot base control on the same items	§6
8	Generate summaries for unseen test conversations	770 predictions on the 6 official test splits, two decoding configurations (run 1, run 3)	§6–§7
9	Save adapter + training configuration	`outputs/kaggle/outputs/baseline_qlora_llama31_8b/` (adapter, checkpoints 200/332, trainer_state, training_config, environments) + private Kaggle dataset `hirthick17/medsum-baseline-adapter`	§8
10	Provide baseline outputs to Person 3	handoff checklist filled in §9; evaluated in `docs/PERSON3_EVALUATION.md`	§9

---

1. Model selection

Decision table in `REVIEW2_BASELINE.md` §1. Summary of the reasoning:

- The baseline must be the *fairest possible comparison* for the Review 3 FaCTS model, which will be built on the same base. The base therefore has to be the one with the best instruction following and the tokenizer/chat template we will keep: **Llama-3.1-8B-Instruct**.
- BioMistral-7B's advantage (biomedical pre-training) is exactly the kind of knowledge the Review 3 fact table supplies explicitly; using it in the baseline would confound the ablation. It remains the no-blocker fallback and ablation A7.
- Access: `meta-llama/Llama-3.1-8B-Instruct` is gated and the team had no approved token by the deadline. `unsloth/Meta-Llama-3.1-8B-Instruct` is a byte-identical, ungated redistribution under the Llama 3.1 Community Licence (verified through the HF API: `gated: false`, same chat template). `training_config.json` records this (`base_model_note`).

2. Environment

where	spec	used for
laptop (Windows 11, RTX 2050 4 GB, Python 3.13, torch 2.11 CPU)	data prep, audits, fact extraction, evaluation, report building	everything that is not an 8B forward pass
Kaggle kernel A `hirthick17/medsum-baseline-qlora`	2× Tesla T4 (2×15 GB, no bf16, no flash-attention), Python 3.12, torch 2.10+cu128, transformers 4.57, peft 0.19, bitsandbytes 0.50, liger-kernel	training

where	spec	used for
Kaggle kernel B `hirthick17/medsum-baseline-infer`	same hardware	validation checks + test-set generation

Kaggle is driven from the terminal (`kaggle kernels push/status/output`, `scripts/fetch_kaggle_run.py`), never from a browser, so every run is reproducible from the repository. The exact `pip freeze` of each kernel is stored as `environment.txt` / `environment_infer.txt` next to the adapter.

3. QLoRA configuration and training pipeline (as run)

```
base          unsloth/Meta-Llama-3.1-8B-Instruct          quantization  NF4, double-quant, fp16 compute
LoRA          r=16, alpha=32, dropout=0.05                targets       q,k,v,o,gate,up,down_proj
trainable     41,943,040 / 8,072,204,288  (0.52 %)         max_seq_len   4096 (1 of 4,265 examples dropped)
batch         1 × grad-accum 16 = 16 examples / step       epochs        2 planned = 534 steps
lr            2e-4, cosine, warmup 3 %                     optimizer     paged AdamW 8-bit
loss          assistant tokens only (prompt masked to -100), EOT appended
memory        Liger fused linear-cross-entropy (never materialises [4096 × 128,256] logits)
parallelism   device_map="balanced" over 2 T4 (naive pipeline; one T4 OOMs in backward at 4k tokens)
guard         stop gracefully at MAX_TRAIN_HOURS so eval + save always fit the 12 h kernel cap
seed          42 (data sampling, shuffling, trainer)
```

Data in: `baseline_train.jsonl` (4,265) / `baseline_val.jsonl` (220) with SHA-256 checked against `train_manifest.json` inside the kernel; the leakage check and the token-length audit are re-run in the kernel and their output is in the log. Prompt format (`REVIEW2_BASELINE.md` §4.2): one system instruction, a per-corpus task instruction (`Write a complete clinical note with the sections …` / `Write a SOAP note …` / `Write the {section_header} section …`), the transcript as the user turn, the note as the assistant turn — rendered with the model's own chat template so training and inference tokenise identically.

4. The training run

kernel / date	`medsum-baseline-qlora`, 17 Sep 2026 22:12 IST → 18 Sep ~05:45 IST
steps	332 of 534 planned (1.24 of 2 epochs); stopped by the 7.5 h wall-clock guard (81 s / step measured)
loss	train 0.73 (step 10) → 0.42–0.45 (steps 280–330), two spikes at steps 270 and 310 (long ACI-Bench examples, grad-norm 1.0); eval 1.118 (100) → **0.9555 (200)** → 0.9614 (300)
selected adapter	`final/` = checkpoint-200 (best eval loss); checkpoint-332 (last, with optimizer state) also kept
adapter sha256	`14e712617f1456b669ef02e6c512e41464dbeee450bbed6245045cb568b69481`
eval loss rising after step 200	the 67 ACI-Bench examples (1.6 % of the mix) are seen ~3× by step 300; train loss keeps falling while eval rises → mild over-fitting to the small full-note slice, hence best-checkpoint selection

The loss curve is in `reports/baseline_qlora_llama31_8b/loss_curve.png` and the full training report (auto-generated, with the §7 validation checks and a per-split table) in `reports/baseline_qlora_llama31_8b/report.pdf`.

5. Issues solved

All nine are documented with symptom → cause → fix in `REVIEW2_BASELINE.md` §6 and dated in `STATUS.md`. The ones that changed the design:

issue	fix that is now in the code
OOM inside `cross_entropy` at 4k tokens (logits `[4096 × 128k]` in fp32 + shifted copy ≈ 4 GB)	Liger fused linear-CE (`use_liger_kernel`), the single biggest memory saving
OOM in backward on one T4 even with Liger (32 × 4k × 4k attention, no flash-attention on sm75)	split across 2 T4 (`device_map="balanced"`), `PipelineTrainer.compute_loss` moves the loss scalar to the `lm_head` device
one kernel cannot train (7.5 h) *and* generate (≈ 5 h) inside 12 h → run 1 killed mid-generation, log lost	training and generation are separate kernels; every stage of `fetch_kaggle_run.py` is idempotent
kernel B mounted the wrong (20-step smoke) adapter because Kaggle mounts the latest *completed* version of kernel A, and the full run was *cancelled* (run 2 evaluated it for 3.8 h without any check failing)	the adapter is uploaded as an explicit dataset; kernel B pins `EXPECTED_ADAPTER_SHA256` and exits in minute 1 otherwise; the fetch script and the report compare the sha before merging/reporting
run 1: 75 % of ACI-Bench notes hit `max_new_tokens=768` in "He has been having some difficulty with X." loops	run 3 decoding: `max_new_tokens=1024`, `repetition_penalty=1.2`, 6-gram block on *generated* tokens only (HF's `no_repeat_ngram_size` would also forbid copying "metformin 500 mg twice a day" from the transcript)

6. Testing the trained model (validation split, 20 items: 10 ACI-Bench + 10 MTS-Dialog)

Kernel B loads base + adapter in a fresh process — this *is* the §7 adapter-reload test — verifies the sha256, generates the 20 validation items with the adapter and again with the adapter disabled (zero-shot base control), and writes `training_config.json`:

system (run 3 decoding)	ROUGE-L ACI-Bench	ROUGE-L MTS-Dialog	length ratio	truncated / 20
adapter (`final/`)	10.5	18.8	1.29	9
zero-shot base (same prompt)	26.6	15.8	2.78	0

Interpretation: on short section tasks the adapter beats the un-tuned base; on full notes the adapter is *worse than the base model under this decoding* and truncates half of the items. This is the long-note failure analysed in §7 and in `docs/PERSON3_EVALUATION.md`.

7. Test-set generation and the decoding comparison

770 notes on the six official test splits, greedy, deterministic. Two configurations were run with the **same adapter** (the run-2 output with the smoke adapter is excluded from every table):

	run 1 (17 Sep, in-kernel)	run 3 (19 Sep, kernel B v2)
`max_new_tokens`	768	1024
`repetition_penalty`	1.1	1.2
n-gram block	none	6-gram, generated tokens only
splits produced	5 of 6 (DialogueSOAP lost to the 12 h cap)	6 of 6
ACI-Bench truncated	30 / 29 / 31 of 40	29 / 28 / 27 of 40
ACI-Bench failure mode	sentence-level **repetition loops**	**runaway comma lists** of hallucinated terms (ROS-style "denies A, B, C, …" that never ends)
MTS-Dialog truncated	9 / 2 of 200	5 / 6 of 200
DialogueSOAP truncated	—	7 / 250

Run 1's predictions were recovered from the cancelled kernel's output (`outputs/kaggle_run1_old_decoding/`) so that both configurations could be evaluated side by side; the numbers are in `docs/PERSON3_EVALUATION.md` and `reports/eval_baseline/comparison.md`.

Root cause of the ACI-Bench degeneration (why decoding alone cannot fix it). The adapter was selected at step 200 (0.75 epoch); only 67 of the 4,265 training examples (1.6 %) are full-length notes, so the model has seen ~50 full-note endings. It writes correct openings (CC, HPI) and then does not know how such a note ends; whatever the decoder forbids (exact loops at 1.1; repeated 6-grams at 1.2) it replaces with the nearest allowed continuation — a list of ever-new medical terms. The zero-shot base model, which has *not* been pushed towards ROS-style enumeration by DialogueSOAP and MTS-Dialog, terminates every note (0/20 truncated). Short outputs (MTS-Dialog sections, SOAP notes) are unaffected because their endings are abundantly represented in training. A fourth generation run with a milder penalty (1.05) was considered and deliberately **not** run: it would spend ~5 GPU-hours on the symptom. The fix is a data-mix change (up-weight ACI-Bench, add its augmented variants, train the full 2 epochs) and belongs to the Review 3 training run, where the fact table also constrains what the model may enumerate.

8. Saved artefacts

```
outputs/kaggle/outputs/baseline_qlora_llama31_8b/
├── final/                        adapter (adapter_config.json, adapter_model.safetensors 168 MB, tokenizer, chat_template.jinja)
├── checkpoints/checkpoint-200/   = final/, with optimizer/scheduler/rng state (resumable)
├── checkpoints/checkpoint-332/   last step, resumable
├── trainer_state.json            loss curve, best step
├── training_config.json          single source of truth: base, quantization, LoRA, data manifest + SHA, training, hardware,
│                                 versions (both kernels), decoding, validation checks (+ adapter sha256), per-split test stats
├── environment.txt / environment_infer.txt   pip freeze of kernel A / kernel B
└── predictions/
    ├── baseline_qlora/<split>.jsonl   six files, 770 lines, schema REVIEW2_BASELINE.md §8.3
    ├── val_check.jsonl                20 validation items, adapter
    └── zeroshot_base/val_check.jsonl  the same 20 items, adapter disabled
outputs/kaggle_run1_old_decoding/…/predictions/   run 1 (768 / 1.1), 5 splits + val_check, same schema
reports/baseline_qlora_llama31_8b/{report.pdf, metrics.json, loss_curve.png, training_config.json, trainer_state.json}
Kaggle: dataset hirthick17/medsum-baseline-adapter (final/ + trainer_state.json), private
```

The adapter is *not* merged into the base weights (a 16 GB artefact that cannot be reloaded in 4-bit without re-quantising); Person 3 and Review 3 reload base + adapter.

9. Handoff to Person 3 (filled in)

```
HANDOFF — plain QLoRA baseline                                  date: 2026-09-21
[x] predictions/baseline_qlora/  6 files, 770 lines total, 0 JSON errors
[x] predictions/zeroshot_base/   1 file (val_check, 20 lines)
```

```
[x] truncated == true per split: 29 / 28 / 27 / 5 / 6 / 7   (ACI B-1, C-2, C-3, MTS chat, MTS sum, SOAP)
[x] training_config.json present; steps_completed 332, best_step 200 match trainer_state.json
[x] train_manifest.json sha256 matches data/processed/baseline_{train,val}.jsonl   (audit: True / True)
[x] final/ adapter reloads in a fresh process (kernel B); sha256 14e71261… == adapter used for generation
[x] no test id appears in baseline_train.jsonl or baseline_val.jsonl                (audit: 0 / 0)
[x] environment.txt + environment_infer.txt present
[x] run 1 predictions (old decoding) recovered for the decoding comparison
[!] known: ACI-Bench outputs degenerate in ~70 % of items under both decodings — flagged per item
    (`truncated`, and `degenerate` in the evaluation) so Person 3 can report with and without them
```

10. Reproduce

```
kaggle kernels push -p kaggle/baseline --accelerator NvidiaTeslaT4     # kernel A: train (~11 h with the 10.5 h guard)
python scripts/fetch_kaggle_run.py                                      # wait, download, upload adapter dataset, push B, wait, download, verif
python scripts/fetch_kaggle_run.py --stage infer --no-wait              # re-download a finished kernel B run (what was done on 21 Sep)
python scripts/make_report.py --run outputs/kaggle/outputs/baseline_qlora_llama31_8b
```

Person 3 — Evaluation Pipeline and Preliminary Baseline Results (Review 2)

Owner: evaluation role. Scope: the evaluation harness (built while Person 2 was training) and the preliminary evaluation of the plain QLoRA baseline. Every number here is produced by `python -m src.eval.run_eval` and stored under `reports/eval_baseline/<label>/`.

#	Task	Where	
1	Build the evaluation pipeline	`src/eval/` — `metrics.py`, `factuality.py`, `run_eval.py`; tests in `tests/test_factuality.py`	§1
2	ROUGE-1 / ROUGE-2 / ROUGE-L, BERTScore vs. reference summaries	`metrics.py`	§1.1
3	Basic factuality checking: hallucinated information, incorrect diagnosis, incorrect medication / dosage, negation errors, numerical errors, missing clinical information	`factuality.py` (on the CFE of `src/cfe/`)	§1.2
4	Preliminary hallucination rate, negation-flip rate, numerical consistency, clinical-information coverage	`run_eval.py` aggregates	§1.3, §3
5	Tables, graphs, example errors	`metrics.md`, `figures/*.png`, `examples.md`, `comparison.md`	§3–§5
6	Preliminary evaluation results for Review 2	this document	§3–§6

---

1. The pipeline

Input is the prediction schema fixed with Person 2 (`REVIEW2_BASELINE.md` §8.3): one JSONL per test split with `source_dialogue`, `reference_summary`, `prediction`, `truncated`, decoding parameters and ids — self-contained, no joins. `run_eval.py` scores every item, aggregates per split, and writes `per_item.jsonl` (every metric and every flagged entity per prediction), `metrics.json/.md`, `examples.md` and `figures/`. `--compare` builds side-by-side tables and plots for all evaluated runs.

1.1 Standard metrics (`src/eval/metrics.py`)

metric	implementation	note
ROUGE-1, ROUGE-2, ROUGE-L, ROUGE-Lsum (F1)	`rouge_score`, Porter stemming on	same configuration as the ACI-Bench / MEDIQA-Chat 2023 official scorers → comparable to published baselines
BERTScore F1	`bert_score`, `roberta-large` layer 17, English baseline rescaling	rescaled values (−1…1); negative means "less similar than a random sentence pair"
length ratio	words(prediction) / words(reference)	0.8–1.2 is healthy
degeneration diagnostics	share of 4-grams that recur in the output; longest comma-separated run; `degenerate = repeated-4-gram share > 0.5 or list ≥ 25 items`	flags the two failure modes seen in the baseline (loops, runaway lists) so every table can be read with and without them

1.2 Factuality checks (`src/eval/factuality.py`)

The transcript, the reference note and the generated note are each converted into facts by the rule-based Clinical Fact Extractor (`src/cfe/`, see `docs/PERSON1_DATA_PIPELINE.md` §7): canonical entity, category, polarity (negation), certainty, section. A generated note is then compared against the **transcript** (faithfulness — the transcript is the only ground truth for what was said) and against the **reference** (coverage and polarity agreement).

report error type	operational definition	output fields
Hallucinated information	an entity (symptom / history / diagnosis / medication / investigation) in the prediction whose canonical form appears nowhere in the transcript; *strict* variant additionally requires absence from the reference (tolerates lexicon misses on the ASR-style dialogue)	`unsupported_vs_source`, `hallucination_rate`, `hallucination_rate_strict`
Incorrect diagnosis	a diagnosis the prediction *asserts* (ASSESSMENT/PLAN/CC context or a diagnostic cue, polarity present) that is neither mentioned as a condition in the transcript nor asserted in the reference; the converse (reference-asserted diagnosis absent from the prediction) is *missing diagnosis*	`incorrect_diagnosis`, `missing_diagnosis`

report error type	operational definition	output fields
Incorrect medication / dosage	a specific drug (class words like *antibiotic* excluded) absent from transcript and reference; a (drug, value, unit) whose drug the transcript names but with a different dose	`incorrect_medication`, `missing_medication`, `dosage_mismatch`
Negation errors	a symptom / history / diagnosis / medication entity present in both prediction and reference with opposite polarity (e.g. reference *denies chest pain*, prediction *reports chest pain*); investigations are excluded because *"troponin negative"* is a result, not an absent test; also computed against the transcript	`negation_flips`, `negation_flip_rate`, `negation_flip_rate_vs_source`
Numerical errors	a number in the prediction (digits, decimals, BP pairs; word numbers in the transcript are converted: *twenty five* → 25) that occurs in neither transcript nor reference	`unsupported_numbers`, `numerical_consistency`
Missing important clinical information	reference entities absent from the prediction, overall and per category; *important* = diagnosis + medication + symptom + history	`missing_by_category`, `clinical_coverage`, `clinical_coverage_important`, `coverage_<category>`

1.3 Preliminary rates

rate	per item	split aggregate
hallucination rate ↓	unsupported entities / prediction entities	item mean (reported) and micro (pooled entities; dominated by long degenerate outputs)
negation-flip rate ↓	polarity conflicts / entities shared with the reference	item mean and micro
numerical consistency ↑	1 − unsupported numbers / prediction numbers	item mean and micro
clinical-information coverage ↑	reference entities found in the prediction / reference entities	item mean, micro, per category

Items with a zero denominator are skipped for that rate. Two MTS-Dialog test items whose dialogue also occurs in training with another section target (data audit §4) are flagged (`flagged_near_dup`); their ROUGE-L is 1.00 / 0.04, i.e. they do not move a 400-item mean.

1.4 Validation of the checker

`tests/test_factuality.py` (8 tests) constructs a faithful note (all rates clean) and then injects each error type into it — a hallucinated diagnosis, a flipped negation, a changed dose, a swapped drug, changed numbers, dropped medications — and asserts that exactly that error is reported. All pass, with the 14 extractor tests. Limitations are in §6.

2. What was evaluated

label	predictions	decoding	splits
run1_rep1.1	`outputs/kaggle_run1_old_decoding/…` (17 Sep, recovered from the cancelled kernel)	greedy, `max_new_tokens` 768, `repetition_penalty` 1.1	ACI-Bench ×3, MTS-Dialog ×2, val (20) — DialogueSOAP lost to the 12 h cap
run3_rep1.2_ngram6	`outputs/kaggle/outputs/baseline_qlora_llama31_8b/…` (19 Sep, kernel B, sha-pinned adapter)	greedy, 1024, 1.2, 6-gram block on generated tokens	all six splits, val (20), **zero-shot base control** on the same 20 val items

Same adapter (`14e71261…`, best eval loss at step 200) in both. Run 2 (smoke adapter) is excluded. BERTScore was computed once per prediction on CPU (roberta-large, ~17 min per run) and cached.

3. Results

3.1 Standard metrics (×100, item means)

split	n	system	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum	BERTScore	len ratio	truncated	degenerate	ROUGE-L excl. trunc./degen.
ACI-Bench taskB-1	40	run 1	29.3	11.4	**17.9**	26.4	−1.8	1.28	30	18	31.2 (n=10)
		run 3	21.9	6.6	11.4	19.3	−13.6	1.28	29	21	19.9 (n=11)
ACI-Bench taskC-2	40	run 1	29.7	12.5	**17.6**	27.3	−1.6	1.21	29	20	29.6 (n=11)
		run 3	20.9	6.2	10.9	18.5	−16.4	1.11	28	25	17.2 (n=11)
ACI-Bench taskC-3	40	run 1	29.7	11.5	**17.4**	27.1	−1.9	1.20	31	17	30.1 (n=8)
		run 3	23.0	7.2	12.3	20.8	−14.8	1.02	27	25	21.2 (n=12)
MTS-Dialog chat-2023	200	run 1	42.2	19.6	**35.8**	35.8	39.6	1.15	9	8	37.2 (n=190)

split	n	system	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum	BERTScore	len ratio	truncated	degenerate	ROUGE-L excl. trunc./degen.
		run 3	34.8	13.7	29.7	29.7	30.7	1.85	5	17	31.7 (n=183)
MTS-Dialog sum-2023	200	run 1	40.0	16.2	**33.5**	33.5	38.8	1.02	2	3	33.7 (n=197)
		run 3	34.1	11.3	28.6	28.6	30.7	1.36	6	12	29.9 (n=188)
DialogueSOAP test (synthetic)	250	run 3	46.7	16.4	**29.7**	38.5	26.3	0.97	7	7	30.3 (n=243)
validation (10 ACI + 10 MTS)	20	run 1 adapter	33.0	12.6	24.0	28.6	20.1	1.07	7	5	28.8 (n=13)
		run 3 adapter	21.0	6.1	14.7	18.3	7.5	1.29	9	9	19.4 (n=11)
		zero-shot base (run 3 decoding)	35.8	11.7	21.2	30.9	16.2	2.71	0	0	21.2 (n=20)

3.2 Factuality (×100, item means; micro rates in `metrics.md`)

split	system	hallucination ↓	halluc. strict ↓	negation-flip ↓	numerical consistency ↑	clinical coverage ↑	coverage (important) ↑	hallucination excl. degenerate
ACI-Bench taskB-1	run 1	**54.7**	51.5	15.1	55.9	48.6	56.5	39.1
	run 3	77.9	75.3	15.1	66.4	46.7	51.7	46.0
ACI-Bench taskC-2	run 1	**46.6**	41.5	12.9	60.9	46.1	53.1	29.2
	run 3	82.6	80.5	17.5	52.2	47.2	53.5	59.6
ACI-Bench taskC-3	run 1	**51.8**	48.1	12.4	55.3	46.7	53.2	34.3
	run 3	78.2	75.3	20.7	71.0	50.8	55.8	53.6
MTS-Dialog chat-2023	run 1	**30.7**	23.7	4.1	78.4	70.2	70.7	27.2
	run 3	44.3	38.8	6.5	80.2	56.6	58.0	38.2
MTS-Dialog sum-2023	run 1	**40.8**	24.4	6.2	87.5	66.0	68.5	41.2
	run 3	56.7	42.7	6.0	78.8	56.8	58.6	53.8
DialogueSOAP test	run 3	48.6	41.0	3.6	74.3	70.7	71.6	47.3
validation (20)	run 1 adapter	42.1	38.9	14.9	79.1	50.1	53.7	29.8
	run 3 adapter	72.9	71.7	6.7	76.9	40.0	45.6	45.1
	zero-shot base	**18.0**	14.2	14.4	73.4	**71.6**	70.5	18.0

Headline (five official test splits pooled, n = 520, item means ×100):

system	ROUGE-L	hallucination ↓	negation-flip ↓	numerical consistency ↑	clinical coverage ↑
plain QLoRA baseline — run 1 decoding (primary)	**30.7**	**40.4**	**8.2**	**71.1**	**61.3**
plain QLoRA baseline — run 3 decoding	25.1	59.3	10.7	72.0	54.0
run 1, clean outputs only (not truncated, not degenerate; n = 416)	35.1	33.8	5.0	77.4	69.3

3.3 Coverage by category (run 1 unless noted, ×100)

split	symptom	medical history	diagnosis	medication	investigation
ACI-Bench (3 splits, range)	55–59	54–62	46–49	40–48	30–37
MTS-Dialog chat / sum	67 / 60	49 / 53	66 / 65	79 / 71	28 / 22
DialogueSOAP (run 3)	73	79	69	61	69
zero-shot base, val (run 3)	70	60	79	78	72

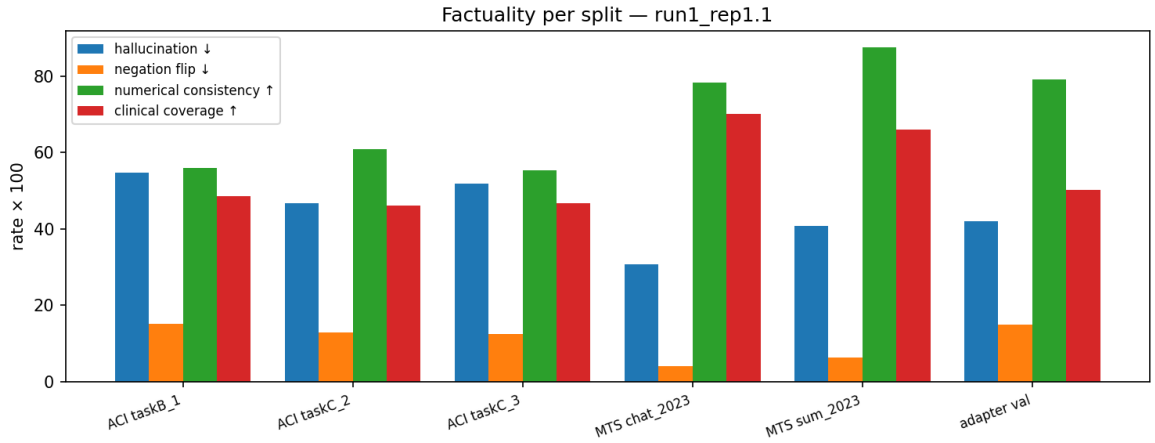
Medications and investigations are the weakest categories on real notes: the baseline drops roughly half of the drugs and two thirds of the ordered tests that the reference note records.

3.4 Error counts (items affected, run 1 / run 3)

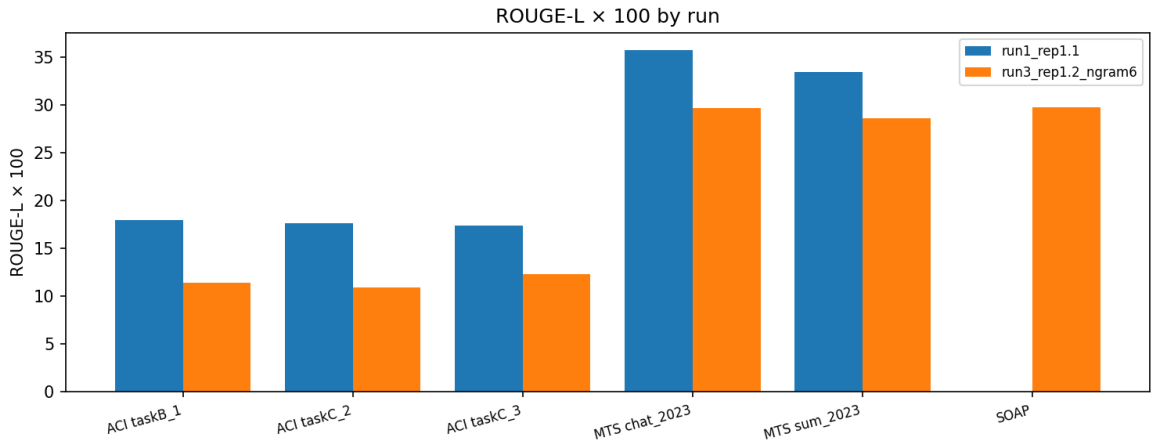
split	n	incorrect dx	missing dx	incorrect med	missing med	dosage mismatch	negation flip	numerical error
ACI-Bench taskB-1	40	1 / 10	22 / 26	2 / 7	23 / 25	0 / 2	24 / 22	26 / 23
ACI-Bench taskC-2	40	4 / 19	24 / 26	2 / 10	27 / 30	0 / 2	23 / 25	31 / 29
ACI-Bench taskC-3	40	7 / 20	25 / 24	4 / 10	27 / 26	4 / 5	22 / 28	31 / 19
MTS-Dialog chat-2023	200	1 / 7	4 / 5	1 / 2	9 / 16	1 / 0	11 / 14	29 / 23
MTS-Dialog sum-2023	200	1 / 6	3 / 6	3 / 9	9 / 12	0 / 1	13 / 12	15 / 23
DialogueSOAP test	250	— / 152	— / 96	— / 49	— / 72	— / 15	— / 43	— / 151

4. Figures

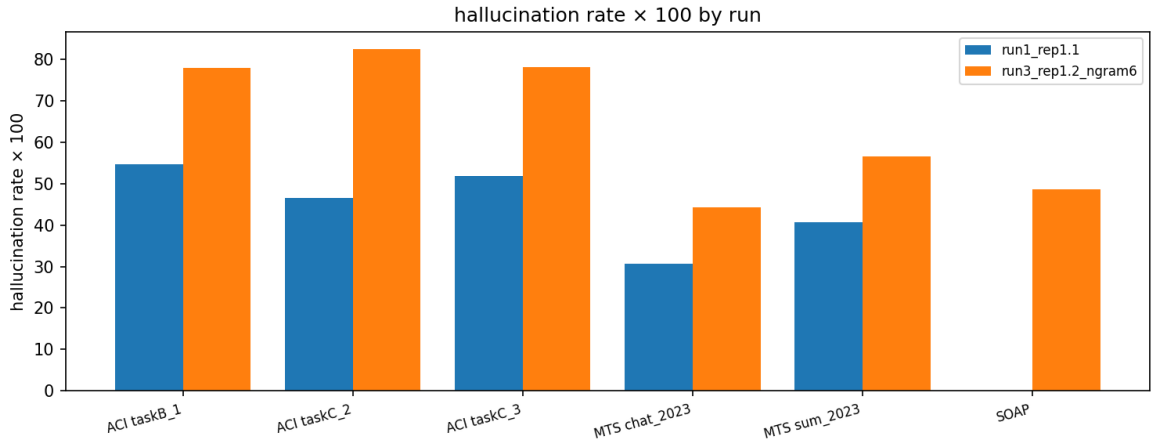
`reports/eval_baseline/<label>/figures/`: `rouge_per_split.png`, `factuality_per_split.png`, `length_ratio_hist.png`, `error_types.png` (% of headline items affected per error type), `rougeL_vs_hallucination.png` (per-item scatter). `reports/eval_baseline/figures/`: `compare_rougeL.png`, `compare_hallucination.png`, `compare_coverage.png`, `compare_negflip.png` (run 1 vs run 3 per split).



factuality per split, run 1



ROUGE-L, run 1 vs run 3



hallucination rate, run 1 vs run 3

5. Example errors

`reports/eval_baseline/<label>/examples.md` holds, per split, the best and worst item and one item per error type with the transcript excerpt, the reference, the generated note and the flags. Three representative cases from the baseline (run 3 unless noted):

Numerical + hallucination — `aci_bench_test_taskB_1_D2N123-aci`**.** Transcript: *"something that I have noticed over the last four to five weeks"*. Generated: *"dysphagia … which started approximately **5-6 months** ago"* (flagged: unsupported number 6) and *"56-year-old **white** male"* (race is never mentioned). The note then degenerates into a 200-item ROS denial list (*denies … sialadenitis, laryngismus, periorbital puffiness, retrobulbar neuritis …*) — 91% of its entities are unsupported.

Incorrect medication — `aci_bench_test_taskB_1_D2N093-virtassist`**.** Generated note lists *testosterone* and *calcium supplement*; neither appears in the transcript, while the transcript's *furosemide, insulin, omeprazole* are missing. 33 numbers in the note (lab values 0.05, 0.7, 1.02, 1.04 …) occur nowhere in the transcript (numerical consistency 0.11).

Negation flip — MTS-Dialog `test_chat_2023_170` (run 1). Reference: *"He denies any weakness, numbness, tingling, dysarthria or diplopia."* Generated: *"He has had some episodes of diplopia. He has had some episodes of dysarthria. … episodes of right-sided numbness."* — three denied findings asserted as present (`negation_flips`: double vision, numbness, slurred speech), inside a loop that also invents syncope, tremor and photophobia.

A checker false positive that changed the metric — MTS-Dialog `test_chat_2023_60`**.** Reference: *"cardiac enzymes were negative x1"*; generated: *"CBC, BMP, cardiac enzymes negative x1"*. The first version of the checker counted *CBC* and *BMP* as negation flips: for a test, "negative" is a *result*, not the absence of the test. Investigations are therefore excluded from the negation-flip metric (it is computed over symptoms, history, diagnoses and medications only).

Degenerate loop (run 1) vs runaway list (run 3) — `aci_bench_test_taskB_1_D2N088-virtassist`**.** Run 1: *"He has been having some difficulty with odynophagia. He has been having some difficulty with globus sensation. …"* (45% of its 4-grams recur; ROUGE-L 17.3, hallucination 66%). Run 3, same item: *"… Denies weight loss, night sweats, chills, abdominal pain, flank pain, urinary frequency, … , Takayasu arteritis, polymyalgia rheumatica …"* (196-item list; ROUGE-L 7.2, 86% of entities unsupported).

Clean output, for contrast — `mts_dialog_test_chat_2023_2`**.** Reference: *"MSK: Negative myalgia, negative joint pain, negative stiffness, negative weakness, negative back pain."* Generated: *"MUSCULOSKELETAL: Negative for myalgias, arthralgias, stiffness, weakness and positive for negative back pain."* — every polarity agrees (0 flips), ROUGE-L 0.54.

6. Preliminary conclusions for Review 2

1. **The plain QLoRA baseline works on short, well-represented targets and fails on long notes.** On MTS-Dialog sections the baseline reaches ROUGE-L 35.8 / 33.5 (published MEDIQA-Chat 2023 systems: ~35–45) with 4.1–6.2% negation flips and 66–70% clinical coverage. On ACI-Bench full notes it reaches 17.4–17.9 (published LED/BART baselines: ~40–45) because 72–78% of outputs hit the token cap: the adapter (selected at step 200, trained on a mix that is only 1.6% full notes) has not learnt how a full note ends. Restricted to the 29 clean ACI-Bench outputs, ROUGE-L is 29.6–31.2 — the model *can* write the note; it cannot stop.

2 **Anti-repetition decoding trades repetition for hallucination.** Run 3 (`repetition_penalty` 1.2 + 6-gram block) removed the loops and moved every metric the wrong way: ACI-Bench ROUGE-L 17.9 → 11.4, hallucination 55 → 78%, MTS-Dialog ROUGE-L 35.8 → 29.7, coverage 70 → 57%. Forbidding the model's preferred (repetitive but grounded) continuation makes it emit *novel* content, which on a clinical note means invented findings. Run 1 is therefore the primary baseline for the results table, and decoding-side repetition control is *not* a factuality tool — this is a concrete argument for fact-constrained generation (Review 3) rather than decoding heuristics.

3 **Fine-tuning reduced factuality relative to the un-tuned base on the validation items.** The zero-shot Llama-3.1-8B-Instruct, given the same prompt, hallucinates 18% of entities and covers 72% of the reference facts, versus 42–73% and 40–50% for the adapter — while being 2.7× too long and scoring lower on ROUGE. The adapter learnt the *form* of a note (length, sections, terse style) faster than the discipline of writing only what was said. This is exactly the gap the FaCTS fact table and ORPO preference tuning target; the zero-shot row is kept as a control in every Review 3 table.

4 **Where the baseline loses information.** Medications (40–48% coverage on ACI-Bench) and investigations (30–37%) are the least covered categories; 23–27 of 40 ACI-Bench notes miss at least one medication the reference records. Symptoms are the best covered (55–59%). Negation flips occur in 55–60% of ACI-Bench notes (12–15% of shared entities) and ~6% of MTS-Dialog sections.

5 **Numbers are unreliable in the generated notes.** Numerical consistency is 55–61% on ACI-Bench (26–31 of 40 notes contain at least one number that appears nowhere in the transcript or reference: invented lab values, altered durations) and 78–88% on MTS-Dialog.

6 **Standard metrics and factuality metrics disagree by design.** ROUGE-L correlates with degeneration (a runaway list still overlaps the reference vocabulary), while the hallucination rate separates the clean outputs from the degenerate ones (39% vs 90% on ACI-Bench). Both must be reported; `rougeL_vs_hallucination.png` shows the spread.

Limitations of this preliminary evaluation

- The factuality checker is lexicon-based: an entity phrased outside the lexicon is invisible to it, and the transcript side (ASR-style, colloquial) is harder to extract from than the note side, which inflates the hallucination rate; the *strict* variant (also accepting the reference) gives the lower bound. The extractor's manual spot-check precision is 87.5% on note facts (`PERSON1_DATA_PIPELINE.md` §7.2), so rates carry roughly ±5 points of extraction noise.
- "Incorrect diagnosis" uses the transcript's condition mentions as support, so a diagnosis the doctor states in the dialogue is never flagged even if the reference disagrees.
- Negation-flip rate is computed on entities shared with the reference; items with no shared entities (degenerate outputs) are excluded from that mean.
- No human evaluation yet (Review 3 reader study, MODEL_PLAN.md §5.3); no confidence intervals (single greedy run, deterministic) — the per-item files allow bootstrapping later.
- DialogueSOAP is synthetic; its numbers are reported for completeness and are not headline numbers.

7. Reproduce

```
python -m src.eval.run_eval --run outputs/kaggle_run1_old_decoding/outputs/baseline_qlora_llama31_8b --label run1_rep1.1 --bertscore
python -m src.eval.run_eval --run outputs/kaggle/outputs/baseline_qlora_llama31_8b --label run3_rep1.2_ngram6 --bertscore
python -m src.eval.run_eval --compare            # reports/eval_baseline/comparison.md + figures/
python -m pytest tests -q
```

Appendix — generated reports

Verbatim copies of the machine-generated Markdown reports referenced above.

Appendix: reports\eval_baseline\comparison.md

Baseline comparison across runs / decoding configurations

All numbers ×100, item means. Runs:

- **run1_rep1.1**: `outputs\kaggle_run1_old_decoding\outputs\baseline_qlora_llama31_8b` · decoding `{"max_new_tokens": 768, "do_sample": false, "repetition_penalty": 1.1}`
- **run3_rep1.2_ngram6**: `outputs\kaggle\outputs\baseline_qlora_llama31_8b` · decoding `{"max_new_tokens": 1024, "do_sample": false, "num_beams": 1, "repetition_penalty": 1.2, "no_repeat_ngram_size_generated_only": 6, "eos_token_id": [128009], "batch": 4, "adapter": "final", "non_quantised_params_fp32": true}`

ROUGE-L ↑

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	17.9	11.4
aci_bench_test_taskC_2	17.6	10.9
aci_bench_test_taskC_3	17.4	12.3
mts_dialog_test_chat_2023	35.8	29.7
mts_dialog_test_sum_2023	33.5	28.6
dialogue_soap_test	—	29.7
val_check	24.0	14.7
zeroshot_base/val_check	—	21.2

BERTScore ↑

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	-1.8	-13.6
aci_bench_test_taskC_2	-1.6	-16.4
aci_bench_test_taskC_3	-1.9	-14.8
mts_dialog_test_chat_2023	39.6	30.7
mts_dialog_test_sum_2023	38.8	30.7
dialogue_soap_test	—	26.3
val_check	20.1	7.5
zeroshot_base/val_check	—	16.2

hallucination ↓

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	54.7	77.9
aci_bench_test_taskC_2	46.6	82.6
aci_bench_test_taskC_3	51.8	78.2
mts_dialog_test_chat_2023	30.7	44.3
mts_dialog_test_sum_2023	40.8	56.7
dialogue_soap_test	—	48.6
val_check	42.1	72.9
zeroshot_base/val_check	—	18.0

negation-flip ↓

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	15.1	15.1
aci_bench_test_taskC_2	12.9	17.5
aci_bench_test_taskC_3	12.4	20.7
mts_dialog_test_chat_2023	4.1	6.5
mts_dialog_test_sum_2023	6.2	6.0
dialogue_soap_test	—	3.6

split	run1_rep1.1	run3_rep1.2_ngram6
val_check	14.9	6.7
zeroshot_base/val_check	—	14.4

numerical consistency ↑

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	55.9	66.4
aci_bench_test_taskC_2	60.9	52.2
aci_bench_test_taskC_3	55.3	71.0
mts_dialog_test_chat_2023	78.4	80.2
mts_dialog_test_sum_2023	87.5	78.8
dialogue_soap_test	—	74.3
val_check	79.1	76.9
zeroshot_base/val_check	—	73.4

clinical coverage ↑

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	48.6	46.7
aci_bench_test_taskC_2	46.1	47.2
aci_bench_test_taskC_3	46.7	50.8
mts_dialog_test_chat_2023	70.2	56.6
mts_dialog_test_sum_2023	66.0	56.8
dialogue_soap_test	—	70.7
val_check	50.1	40.0
zeroshot_base/val_check	—	71.6

length ratio

split	run1_rep1.1	run3_rep1.2_ngram6
aci_bench_test_taskB_1	1.28	1.28
aci_bench_test_taskC_2	1.21	1.11
aci_bench_test_taskC_3	1.20	1.02
mts_dialog_test_chat_2023	1.15	1.85
mts_dialog_test_sum_2023	1.02	1.36
dialogue_soap_test	—	0.97
val_check	1.07	1.29
zeroshot_base/val_check	—	2.71

Truncated / degenerate outputs

split	run1_rep1.1 trunc / degen	run3_rep1.2_ngram6 trunc / degen
aci_bench_test_taskB_1	30 / 18 (n=40)	29 / 21 (n=40)
aci_bench_test_taskC_2	29 / 20 (n=40)	28 / 25 (n=40)
aci_bench_test_taskC_3	31 / 17 (n=40)	27 / 25 (n=40)
mts_dialog_test_chat_2023	9 / 8 (n=200)	5 / 17 (n=200)
mts_dialog_test_sum_2023	2 / 3 (n=200)	6 / 12 (n=200)
dialogue_soap_test	—	7 / 7 (n=250)
val_check	7 / 5 (n=20)	9 / 9 (n=20)
zeroshot_base/val_check	—	0 / 0 (n=20)

Appendix: reports\data_audit\data_audit.md

Data audit — processed corpora

Generated 2026-09-21 22:48. Training window 4096 tokens.

1. Records, unusable samples, duplicates, lengths

corpus	split	n	unusable	exact dup (within)	src words p50/p95/max	tgt words p50/p95/max	prompt tok p50/p95/max	total tok p50/p99/max	> 4096 tok
aci_bench	train	67	0	0	1240/1960/3050	404/620/884	1619/2411/3635	2195/4129/4405	1
aci_bench	validation	20	0	0	1241/1712/1789	435/612/829	1572/2182/2274	2239/3026/3030	0
aci_bench	test_taskB_1	40	0	0	1098/1905/2392	397/653/785	1429/2402/2885	1938/3913/4079	0
aci_bench	test_taskC_2	40	0	0	1261/2414/2681	430/733/884	1590/3028/3275	2135/4312/4461	1
aci_bench	test_taskC_3	40	0	0	1297/2138/2707	419/641/771	1641/2718/3220	2224/3969/4287	1
mts_dialog	train	1201	131	2	64/339/1509	14/150/1020	169/533/2254	193/1157/3681	0
mts_dialog	validation	100	10	0	54/304/630	15/138/295	155/494/872	187/1010/1152	0
mts_dialog	test_chat_2023	200	14	0	66/312/608	14/154/403	169/485/853	197/834/1358	0
mts_dialog	test_sum_2023	200	13	1	59/330/978	12/177/565	162/533/1443	183/1067/2121	0
dialogue_soap	train	9250	0	0	420/494/604	246/312/398	688/794/946	1050/1292/1491	0
dialogue_soap	validation	500	0	0	418/492/559	245/312/384	690/790/876	1052/1276/1315	0
dialogue_soap	test	250	0	0	422/496/550	243/315/340	698/790/859	1052/1289/1314	0

2. Near-duplicates (SimHash, 3-word shingles, Hamming ≤ 3)

corpus	pairs within a split	pairs across splits	pairs involving a test split
aci_bench	0	0	0
mts_dialog	3	9	7
dialogue_soap	0	0	0

3. Leakage checks (baseline_train + baseline_val vs. the six test splits)

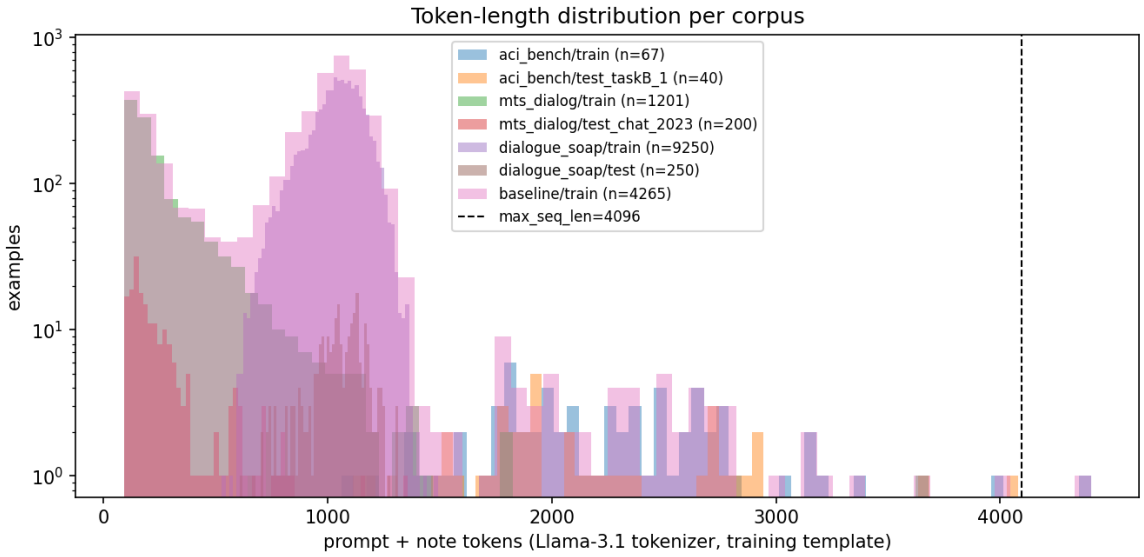
- id overlap train/val: 0, train/test: 0, val/test: 0
- exact source-dialogue overlap with test: 0 (after make_baseline_split.py dropped 3: ['mts_dialog_train_989', 'mts_dialog_train_926', 'mts_dialog_train_901'])
- exact target-note overlap with test: 131 [('mts_dialog_train_142', 'mts_dialog_test_chat_2023_106'), ('mts_dialog_train_388', 'mts_dialog_test_sum_2023_65'), ('mts_dialog_train_868', 'mts_dialog_test_sum_2023_150'), ('mts_dialog_train_725', 'mts_dialog_test_sum_2023_65'), ('mts_dialog_train_109', 'mts_dialog_test_sum_2023_180')]
- near-duplicate source dialogues (mix vs test, Hamming ≤ 3): 2
- train_manifest.json SHA-256 matches on disk: train=True, val=True
- **status: passed**

4. Baseline mix

- train: {'n': 4265, 'mix': {'dialogue_soap': 3000, 'mts_dialog': 1198, 'aci_bench': 67}, 'total_tokens': {'p50': 992, 'p90': 1180, 'p95': 1234, 'p99': 1991, 'p100': 4405}, 'over_max_seq_len': 1, 'over_max_seq_len_ids': ['aci_bench_train_D2N021-virtscribe']}
- val: {'n': 220, 'mix': {'aci_bench': 20, 'mts_dialog': 100, 'dialogue_soap': 100}}
- test: {'aci_bench/test_taskB_1': 40, 'aci_bench/test_taskC_2': 40, 'aci_bench/test_taskC_3': 40, 'dialogue_soap/test': 250, 'mts_dialog/test_chat_2023': 200, 'mts_dialog/test_sum_2023': 200}

5. Lengthy conversations

The training window is 4096 tokens (prompt + note). Kernel A *drops* any example whose rendered length exceeds it rather than truncating (truncating a note teaches the model to stop early; truncating a transcript removes facts the note still asserts). At inference the prompt-only limit is the base model's 128k context, so no test conversation is cut. Counts of dropped examples per split are in the table above (`> 4096 tok`).



token length histogram

Appendix: reports\data_audit\cfe_stats.md

Clinical fact extraction — corpus statistics

Generated 2026-09-21 23:54; total 590.7 s on CPU.

split	n	src facts/doc	ref facts/doc	ref negation rate	src uncertain rate	ref entities found in src	dosage→drug link	ms/rec
aci_bench/train	67	64.2	58.9	0.132	0.197	0.839	0.857	120.6
aci_bench/validation	20	63.9	58.5	0.132	0.193	0.847	0.914	117.4
aci_bench/test_taskB_1	40	60.1	56.5	0.139	0.199	0.832	0.931	118.2
aci_bench/test_taskC_2	40	69.2	60.2	0.136	0.206	0.836	0.833	130.2
aci_bench/test_taskC_3	40	68.8	64.5	0.136	0.211	0.82	0.861	132.8
mts_dialog/train	1201	5.5	4.4	0.201	0.149	0.776	0.58	10.2
mts_dialog/validation	100	5.1	4.6	0.257	0.135	0.748	1.0	8.7
mts_dialog/test_chat_2023	200	5.5	4.4	0.257	0.133	0.791	0.571	9.6
mts_dialog/test_sum_2023	200	5.1	4.4	0.181	0.14	0.726	0.75	10.1
dialogue_soap/train	9250	37.2	37.4	0.056	0.084	0.839	0.238	53.1
dialogue_soap/validation	500	38.0	37.9	0.055	0.088	0.84	0.25	75.1
dialogue_soap/test	250	37.3	37.7	0.057	0.084	0.842	0.277	73.1

Category mix in the reference notes (all splits)

split	symptom	medical_history	diagnosis	medication	dosage	investigation	treatment	numeric
aci_bench/train	987	208	342	393	147	612	984	272
aci_bench/validation	296	45	113	105	35	184	292	99
aci_bench/test_taskB_1	587	101	220	216	72	289	596	178
aci_bench/test_taskC_2	575	118	249	249	108	317	621	171
aci_bench/test_taskC_3	682	124	235	277	108	361	641	151
mts_dialog/train	1607	905	352	466	50	296	1059	591
mts_dialog/validation	166	82	25	41	8	15	85	37
mts_dialog/test_chat_2023	291	123	75	87	7	51	149	96
mts_dialog/test_sum_2023	204	166	65	89	16	50	183	102
dialogue_soap/train	39426	17564	43849	23357	15544	66617	118793	21049
dialogue_soap/validation	2199	932	2413	1324	859	3661	6404	1167
dialogue_soap/test	1123	480	1203	662	423	1764	3225	533

Appendix: STATUS.md

TL;DR (updated 21 Sep, night) — Review 2 closed

- **Run 3 (kernel B v2, sha-pinned adapter) completed on 19 Sep 06:32 UTC.** The laptop poller had died at 0.84 h, so nothing was downloaded until 21 Sep; `fetch_kaggle_run.py --stage infer --no-wait` then downloaded it, verified `adapter sha256 14e71261… == final/`, merged it and built `reports/baseline_qlora_llama31_8b/report.pdf`. All 6 splits / 770 predictions + `training_config.json` + the zero-shot control exist. **No Kaggle kernel is running; no further GPU runs are planned for Review 2.**
- **The new decoding did not fix ACI-Bench.** 29 / 28 / 27 of 40 notes still hit the 1,024-token cap — the repetition loops of run 1 became runaway comma-lists of hallucinated terms (see `docs/PERSON3_EVALUATION.md`, "Degenerate output example"). MTS-Dialog (5 / 6 of 200) and DialogueSOAP (7 / 250) are fine. Root cause is the data mix / early best checkpoint, not decoding (`docs/PERSON2_MODEL_TRAINING.md` §7); a 4th decoding run was considered and not run.
- Run 1's predictions (old decoding, same adapter) were re-downloaded from the cancelled kernel A into `outputs/kaggle_run1_old_decoding/` so both decodings are evaluated side by side.
- **Person 1 and Person 3 deliverables are done on the laptop (CPU only):** data audit (`reports/data_audit/`), rule-based clinical fact extractor (`src/cfe/`, 22 unit tests) and the facts dataset (`data/processed/facts/`, 12,108 records), evaluation pipeline (`src/eval/`: ROUGE-1/2/L, BERTScore, hallucination / negation-flip / numerical-consistency / coverage, tables, figures, example errors) with results in `reports/eval_baseline/`. Documentation: `docs/`.

TL;DR (19 Sep, morning — superseded by the block above)

- The QLoRA baseline **trained successfully** on Kaggle on 17 Sep (Llama-3.1-8B-Instruct, 4-bit, LoRA r=16, 332 steps / 1.24 epochs, best eval loss 0.9555). The adapter is saved and downloaded to `outputs/kaggle/outputs/baseline_qlora_llama31_8b/final/` (sha256 `14e71261…`).
- Run 1 (17–18 Sep, single kernel) was **killed by Kaggle's 12 h limit** mid-generation; we split training and generation into two kernels (below).
- **Run 2 (kernel B, 18 Sep 20:20 → 19 Sep 00:10 IST) completed cleanly — but with the wrong adapter.** Kaggle mounted kernel A's latest *completed* version, which is the 20-step smoke run, not the cancelled full run. All 770 predictions in that run come from a near-untrained adapter. Nothing in the pipeline failed; the adapter sha256 in the log gave it away.
- Fix (done 19 Sep morning, in the repo): the 17 Sep adapter is uploaded as a private Kaggle dataset; kernel B **pins the adapter's sha256** and refuses to run anything else; the fetch script and the report both verify the sha before merging/reporting. Kernel B must be re-run (~4 h GPU) — that is the only thing between us and the handoff.
- Silver lining: run 2 proves the **decoding fix works** — 2 / 770 truncated (run 1: 71 / 520), no repetition loops, ACI-Bench length ratio ≈ 1.0.
- Impact: handoff to evaluation (Person 3) moves to **19 Sep evening** at the earliest (kernel B re-run ≈ 4 h from push). Buffer day (20 Sep) unchanged.

What happened, step by step

17 Sep 22:12 IST — full run starts on Kaggle (`hirthick17/medsum-baseline-qlora`, 2× T4). One script did everything: train → validation checks → generate on all 6 test splits → write `training_config.json`.

~05:45 IST 18 Sep — training finishes. The script's own wall-clock guard stopped it at 7.5 h, step 332 of 534 (epoch 1.24 of 2). Best eval loss 0.9555 at step 200; the saved `final/` adapter is that best checkpoint. Adapter, checkpoints and `trainer_state.json` were written to disk.

~06:00–10:00 IST — generation. Model reloaded, 20 val items and then the test splits generated in order: ACI-Bench ×3 (40 each), MTS-Dialog ×2 (200 each) — all written to disk.

~10:12 IST — Kaggle kills the kernel at the 12 h mark while it was on the 6th and largest split, `dialogue_soap_test` (250 items). Status: `CANCEL_ACKNOWLEDGED`. Everything after that point never happened: the 6th split, the adapter-reload test, `training_config.json`. Kaggle also **discards the log of a cancelled run**, so the printed metrics (peak VRAM, val checks) are gone.

Local side. The script that was supposed to wait for the run and download it (`scripts/fetch_kaggle_run.py`) stopped at 00:34 IST — the laptop slept — so nothing was downloaded until the evening of 18 Sep. That cost us most of a working day before anyone saw the run had been cancelled.

Why the Kaggle run "fails"

Three separate things, in order of importance:

1. One kernel, 12 h cap, ~13 h of work

stage	measured time on 2× T4
pip install + model load + tokenising	~15 min
training, 332 steps at 81 s/step	7.5 h (the guard)
3 evaluations on 220 val items	14 min
model reload for generation	~5 min
540 generations (ACI-Bench notes ≈ 720 tokens each)	~3.5 h
remaining: 250 DialogueSOAP items + config	**never ran — cap hit**

Training is slow because of the hardware, not the code: T4s have no bf16 and no FlashAttention; 4-bit dequantisation and gradient checkpointing each cost ~1.3×; Kaggle's two T4s run as a naive pipeline (one works while the other waits); median example ≈ 900 tokens at batch size 1. That gives **~5 s per training example → 2 epochs over 4,265 examples ≈ 12 h on their own.** REVIEW2_BASELINE.md §4.6 predicted 9–14 h on a T4; the single-kernel layout did not honour that.

2. No log, no config → the report pipeline could not run

`scripts/make_report.py` builds the Review 2 PDF from `training_config.json`, which is the last thing the script writes. Cancelled → not written → no report, even though 90 % of the results existed on disk.

3. Output quality: repetition loops on long notes (found only after inspecting predictions)

On ACI-Bench (full clinical notes, 400–800 words), **75 % of outputs hit the 768-token cap** (30, 29, 31 of 40), most in loops like:

> He has been having some difficulty with odynophagia. He has been having some difficulty with > globus sensation. He has been having some difficulty with regurgitation. …

Cause: `repetition_penalty=1.1` is too weak for long notes, and only 67 of the 4,265 training examples (1.6 %) are full-length notes, so the model has barely learnt how such a note ends. Short MTS-Dialog sections are fine (9/200 and 2/200 truncated).

Run 1 numbers (ROUGE F1 × 100, stemmed; first look only — Person 3's evaluation supersedes):

split	n	R-1	R-2	R-L	len ratio	truncated
aci_bench taskB-1 / C-2 / C-3	40 each	29.3–29.7	11.4–12.5	**17.4–17.9**	1.18–1.26	29–31 / 40
mts_dialog chat-2023	200	42.2	19.6	**35.8**	1.15	9
mts_dialog sum-2023	200	40.0	16.2	**33.5**	1.02	2
dialogue_soap test	0 / 250	—	—	—	—	lost

Run 2 (18 Sep, kernel B): completed, wrong adapter

What the Kaggle terminal shows. `hirthick17/medsum-baseline-infer` — status `COMPLETE`, 3.82 h wall, peak 10.5 GB / GPU, all six splits written, `training_config.json` written. Log (kept at `outputs/kaggle_run2_smoke_adapter/medsum-baseline-infer.log`):

```
[env] adapter at /kaggle/input/notebooks/hirthick17/medsum-baseline-qlora/outputs/baseline_qlora_llama31_8b_smoke/final
model loaded | adapter params 41.9M / 8.07B | adapter sha256 f431c7434ba6e411 | VRAM 2.8 + 4.6 GB
final/ adapter matches checkpoint(s) none mounted | best eval step 20 (0.9539413452148438)
```

The adapter we hold locally is `14e712617f1456b6…` (332 steps, best step 200). `f431c743…` is the smoke-test adapter: 20 optimizer steps over the 64 longest training examples. The `training_config.json` it produced says `max_steps: 20, epochs_completed: 5.0` for the same reason.

Why. Kernel B lists kernel A in `kernel_sources`. Kaggle mounts the latest *successfully completed* version of A. The 17 Sep full run ended `CANCEL_ACKNOWLEDGED`, so the version before it — the smoke run — is what got mounted. `kaggle kernels output` *does* return a cancelled run's files, which is why we have the real adapter on the laptop but the kernel did not.

Why nothing caught it. Every check inside kernel B is self-consistent: the adapter loaded, it matched the (smoke) `trainer_state.json` it found next to it, outputs were non-empty and untruncated. No check compared what the kernel ran against what *we* intended to run.

What run 2 does tell us (`reports/run2_smoke_adapter_18sep/report.pdf`, red banner):

split	n	R-L (smoke adapter)	R-L run 1 (real adapter, old decoding)	len ratio	truncated
aci_bench taskB-1 / C-2 / C-3	40 each	32.4 / 32.3 / 31.2	17.4–17.9	0.97–1.05	1 / 0 / 1
mts_dialog chat-2023	200	24.5	35.8	3.31	0
mts_dialog sum-2023	200	24.4	33.5	3.13	0
dialogue_soap test	250	25.6	—	0.57	0

- Decoding fix confirmed: the repetition loops are gone, ACI-Bench notes end at the right length, 2 of 770 truncated. This holds regardless of the adapter.
- The MTS-Dialog collapse (35.8 → 24.5, outputs 3× too long) and short SOAP notes are what a 20-step adapter looks like — a reminder that the 332-step adapter is doing real work.
- Zero-shot base control on the same 20 val items: ROUGE-L 21.2 vs 26.7 for the smoke adapter.
- **None of the run-2 numbers are baseline numbers.** They are kept under `outputs/kaggle_run2_smoke_adapter/` and `reports/run2_smoke_adapter_18sep/`, both labelled.

Fix (in the repo, 19 Sep morning).

where	change
`kaggle/adapter/dataset-metadata.json` + `fetch_kaggle_run.py --stage adapter`	the 17 Sep `final/` + `trainer_state.json` are uploaded as the private dataset `hirthick17/medsum-baseline-adapter` (~185 MB); kernel B lists it in `dataset_sources`
`kaggle/infer/baseline_infer.py`	prefers `/kaggle/input/datasets/**/final/` over the notebook mount; refuses a `*_smoke` path in full mode; `EXPECTED_ADAPTER_SHA256 = 14e71261…` — any other adapter stops the kernel in minute 1, not hour 4
`scripts/fetch_kaggle_run.py`	kernel B's output is downloaded to a staging dir and merged into `outputs/kaggle/` only if the sha kernel B hashed equals the local `final/adapter_model.safetensors` (run 2 would otherwise have overwritten the real `trainer_state.json`)
`scripts/make_report.py`	first line of the §10 handoff checklist is now "adapter used for generation == final/ (sha256)"; a mismatch, or a `*_smoke` checkpoint under `mode=full`, puts a red **NOT THE BASELINE** banner on page 1; auto-detect refuses when several runs exist under `outputs/`

Next step: `python scripts/fetch_kaggle_run.py --stage adapter` (upload, ~5 min), then `python scripts/fetch_kaggle_run.py --stage infer --push` (≈ 4 h; downloads, verifies, builds the report). Kaggle GPU quota used this week: ≈ 12 h (run 1) + 3.8 h (run 2) of 30 h; the re-run takes ≈ 4 h more.

How we solved it (all in the repo as of 18 Sep)

Two kernels instead of one.

kernel	Kaggle id	does	budget
A `kaggle/baseline/`	`hirthick17/medsum-baseline-qlora`	trains only → `final/`, `trainer_state.json`, `train_summary.json`	guard at 10.5 h (≈ 1.75 epochs), save fits easily
B `kaggle/infer/`	`hirthick17/medsum-baseline-infer`	loads the adapter from the dataset `hirthick17/medsum-baseline-adapter` (sha256-pinned; see Run 2), runs the §7 checks + zero-shot base control, generates all 6 splits, writes `training_config.json`	~4 h (measured 3.8 h)

Both write the same output path, so downloading both gives one merged folder that the report script reads. **B can be re-run in ~4 h to change decoding without retraining.** (The original plan had B mount A's output directly via `kernel_sources`; see Run 2 for why that was replaced by an explicit adapter dataset.)

Decoding fixes in kernel B (REVIEW2_BASELINE.md §8.2):

- `max_new_tokens` 768 → **1024** — ACI-Bench references reach ~1,020 tokens; 768 was clipping ~15 % of correct-length notes even without loops.
- `repetition_penalty` 1.1 → **1.2**.
- **6-gram repeat block on generated tokens only.** HF's built-in `no_repeat_ngram_size` also forbids copying a 6-gram from the transcript ("metformin 500 mg twice a day"), which a faithful note must do — so we use a custom processor that only looks at what the model has generated.

Tooling:

- `scripts/fetch_kaggle_run.py` now drives both kernels (wait for A → download → push B → wait → download → report), every stage idempotent — if the poller dies again, rerun it, nothing is lost.
- `scripts/make_report.py` reads the merged output and shows the decoding actually used.
- B rebuilds the training half of `training_config.json` from `trainer_state.json` when given pre-split output (the 17 Sep run has no `train_summary.json`).

What this means for you

Person 3 (evaluation): expect the handoff (`outputs/…/predictions/baseline_qlora/*.jsonl`, 6 files / 770 lines, plus `training_config.json` and the PDF) once kernel B has been re-run with the right adapter — target evening of 19 Sep. Do not use anything from `outputs/kaggle_run2_smoke_adapter/`. Schema unchanged (§8.3); two new fields inside `decoding` (`no_repeat_ngram_size_generated_only`, `num_beams`). The `truncated` flag is still there; expect far fewer `true` on ACI-Bench. There is also a `predictions/zeroshot_base/val_check.jsonl` — the un-adapted base model on the same 20 val items, a free control row for the results table.

Everyone: the adapter we hand over is the **1.24-epoch** one from 17 Sep. Retraining with the new 10.5 h guard would give ~1.75 epochs and a properly completed cosine schedule, but costs ~11 h + 2.5 h of the 30 h/week Kaggle GPU quota (run 1 used ~12 h; B uses ~2.5 h). Proposal: ship DA2 on the 17 Sep adapter with the fixed decoding; retrain only if the quota and calendar allow before the 20th. Objections → say so today.

Review 3 note: the long-note weakness is a data-mix issue (1.6 % ACI-Bench) that no decoding setting fixes. Up-weighting ACI-Bench / adding its augmented splits goes on the Review 3 list.

Timeline vs plan

Appendix A	planned	actual
Full training run	16 Sep	17 Sep 22:12 → done ~05:45 18 Sep
§7 checks on val	17 Sep	18 Sep (in-run; re-done in kernel B)
§8 generation, 6 splits	18 Sep	5/6 on 18 Sep morning (run 1); all 6 on 19 Sep 00:10 but wrong adapter (run 2); re-run 19 Sep
§10 handoff	19 Sep	19 Sep evening (best case)
Buffer / submission	20 Sep	unchanged